

Chapter 15 -- Creating Subclasses: Building Semantic Taxonomy in Ontology

- [15.1 Chapter Introduction -- Why Subclasses Matter](#)
- [15.2 From Flat Classes to Semantic Taxonomy](#)
- [15.3 Understanding Subclasses in OWL](#)
- [15.4 Interesting Reading -- The Mathematical Meaning of Subclasses](#)
- [15.5 Creating Subclasses in Protégé](#)
- [15.6 Exercise 14 -- Creating Subclasses of **Pizza**](#)
- [15.7 Semantic Inheritance](#)
- [15.8 Interesting Reading -- Semantic Inheritance vs OOP Inheritance](#)
 - [15.8.1 Inheritance in Object-Oriented Programming](#)
 - [15.8.2 Semantic Inheritance in Ontology](#)
 - [15.8.3 Key Difference](#)
- [15.9 Subclass vs. Instance -- Common Beginner Confusion](#)
- [15.10 Graph Database Perspective -- Hierarchy in Neo4j vs. Ontology](#)
- [15.11 EKA Perspective -- Taxonomy as Semantic Compression](#)
- [15.12 Key Concepts](#)
- [15.13 Chapter Summary](#)
- [15.14 Protégé Skills Checklist](#)
- [15.15 Looking Ahead](#)

15.1 Chapter Introduction -- Why Subclasses Matter

After completing Chapter (14), you have entered a much deeper stage of ontology engineering.

The previous chapter introduced:

- semantic restrictions,
- logical requirements,
- reasoning boundaries, and
- formal inference behavior.

Ontology was no longer merely about representing knowledge structures.

It became increasingly concerned with:

semantic logic.

In this chapter, we return to a concept that may initially appear much simpler:

subclasses.

Inside Protégé, creating a subclass is mechanically straightforward.

You typically:

1. select a parent class,

2. click *Add subclass*, and
3. assign a class name.

However, the semantic significance of subclass modeling is far greater than its simple user interface.

Subclasses provide the foundation for:

- taxonomy,
- specialization,
- inheritance, and
- classification reasoning.

Without subclass hierarchies, ontology quickly becomes:

- flat structure,
- difficult to navigate, and
- semantically weak.

Subclass modeling therefore represents one of the most fundamental building blocks of ontology engineering.

15.2 From Flat Classes to Semantic Taxonomy

Consider a knowledge model where all concepts exist at the same level:

```
Pizza
NamedPizza
VegetarianPizza
SpicyPizza
MargheritaPizza
```

Although these concepts are present, the model lacks semantic structure.

Nothing explicitly indicates:

- which concepts are more generalized
- which are more specialized
- how they relate conceptually

This is known as:

flat classification.

Flat classification may be sufficient for small datasets, but it scales poorly when knowledge becomes complex.

Ontology addresses this problem using:

taxonomy.

A taxonomy organizes concepts hierarchically.

```
./
├── Pizza
│   ├── NamedPizza
│       ├── AmericanaPizza
│       ├── MargheritaPizza
│       └── SohoPizza
```

This hierarchy immediately conveys semantic relationships message to the viewers.

Taxonomy therefore provides not only organization, but also:

semantic context.

This context is essential for reasoning.

15.3 Understanding Subclasses in OWL

In ontology engineering, subclass relationships are commonly expressed using:

is-a relationships.

For example: `MargheritaPizza is a NamedPizza`

This means:

`MargheritaPizza is a type of NamedPizza`

Not:

`MargheritaPizza contains NamedPizza`

Nor:

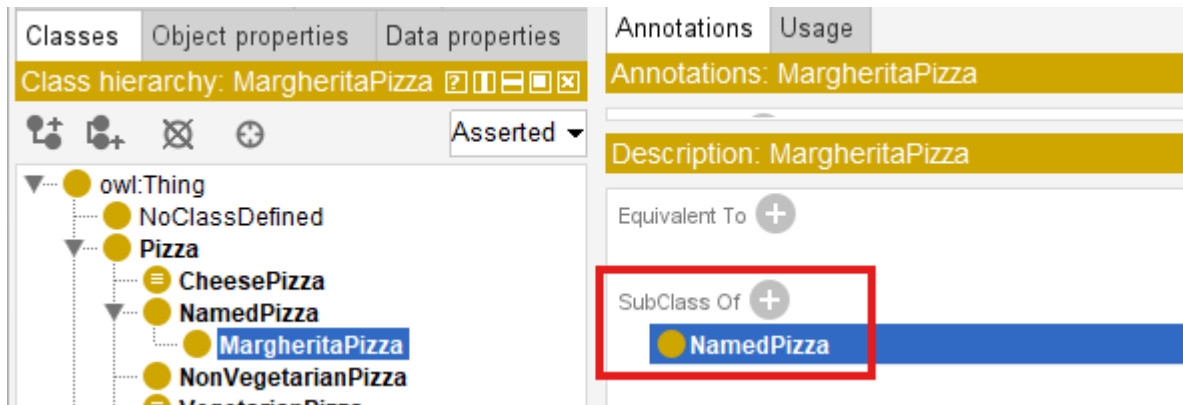
`MargheritaPizza1 references NamedPizza`

Instead, it expresses:

semantic specialization.

In OWL, subclass relationships are represented through `SubClassOf`.

An example: `MargheritaPizza SubClassOf NamedPizza`



Semantically, this states:

every **MargheritaPizza** is also a **NamedPizza**.

This is a strong logical assertion.

Subclass relationships therefore carry formal meaning, not merely visual hierarchy.

15.4 Interesting Reading -- The Mathematical Meaning of Subclasses

To fully understand subclasses, it helps to view them through mathematics.

Ontology class semantics closely align with:

set theory.

A class can be interpreted as:

a set of individuals sharing common characteristics.

Now suppose:

P = set of all pizzas

We can define:

N = set of all named pizzas

This means:

every element of N is also an element of P .

This is the formal meaning of subclass.

More generally:

If :

A = parent class

B = subclass

Then :

$B \sqsubseteq A$

Equivalent logical notation is:

$$\forall x (x \in B \rightarrow x \in A)$$

This reads as:

For every x , if x belongs to subclass B , then x must also belong to class A .

This mathematical definition explains why subclass reasoning works.

Suppose:

`myPizza` $\in$ `MargheritaPizza`

And ontology defines:

`MargheritaPizza` $\sqsubseteq$ `NamedPizza`

`NamedPizza` $\sqsubseteq$ `Pizza`

Then by transitivity:

`myPizza` $\in$ `Pizza`

The reasoner performs exactly this type of logical inference automatically.

This is one of the earliest examples showing how ontology enables:

machine reasoning through formal mathematics.

In other words, subclass hierarchy is not merely a tree.

It is formally defined semantic inclusion system.

15.5 Creating Subclasses in Protégé

In the `Pizza` tutorial, the next step is to expand the ontology by creating subclasses under existing classes.

The workflow in Protégé is straightforward.

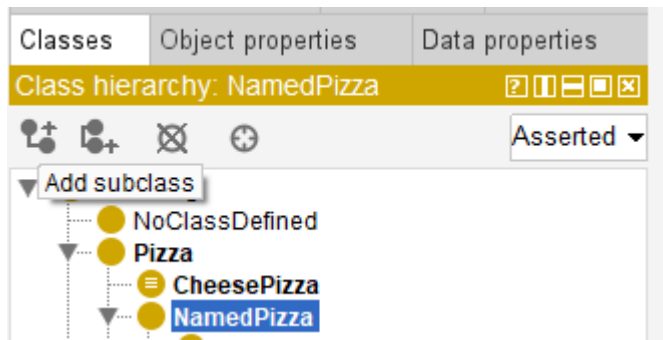
First: select a parent class.

For example: `NamedPizza`

Then choose:

Add subclass

Protégé creates a new child class.



Although visually simple, this action generates a formal OWL axiom:

```
NewClass SubClassOf NamedPizza
```

This expands the ontology's semantic hierarchy.

Each new subclass increases the expressiveness of the ontology.

As more specialized pizza types are introduced, the ontology becomes better suited for:

- classification,
- reasoning, and
- semantic discovery.

15.6 Exercise 14 -- Creating Subclasses of **Pizza**

Having understood the conceptual meaning of subclasses, we now return to the practical ontology engineering workflow introduced in Michael DeBellis' **Pizza** tutorial.

Exercise 14 marks an important milestone in the tutorial because it is the first time you begin transforming the **Pizza** ontology from a relatively flat conceptual model into a meaningful semantic hierarchy.

The objective of Exercise 14 is straightforward:

create subclasses under the **Pizza** class to represent more specialized pizza categories.

Specifically, the tutorial introduces two new classes:

```
NamedPizza
MargheritaPizza
```

The resulting hierarchy becomes:

```

/
└ Pizza
  └ NamedPizza
    └ MargheritaPizza
```

At first glance, this may appear to be a simple taxonomy expansion.

However, semantically, something much more important is happening.

By introducing `NamedPizza`, the ontology gains an intermediate abstraction layer.

This layer acts as a semantic grouping class for pizzas that represent recognizable recipe types or named menu items.

Examples includes:

- `MargheritaPizza`
- `AmericanaPizza`
- `SohoPizza`

(You may find other two type of pizzas in `pizza-ebook.rdf`.)

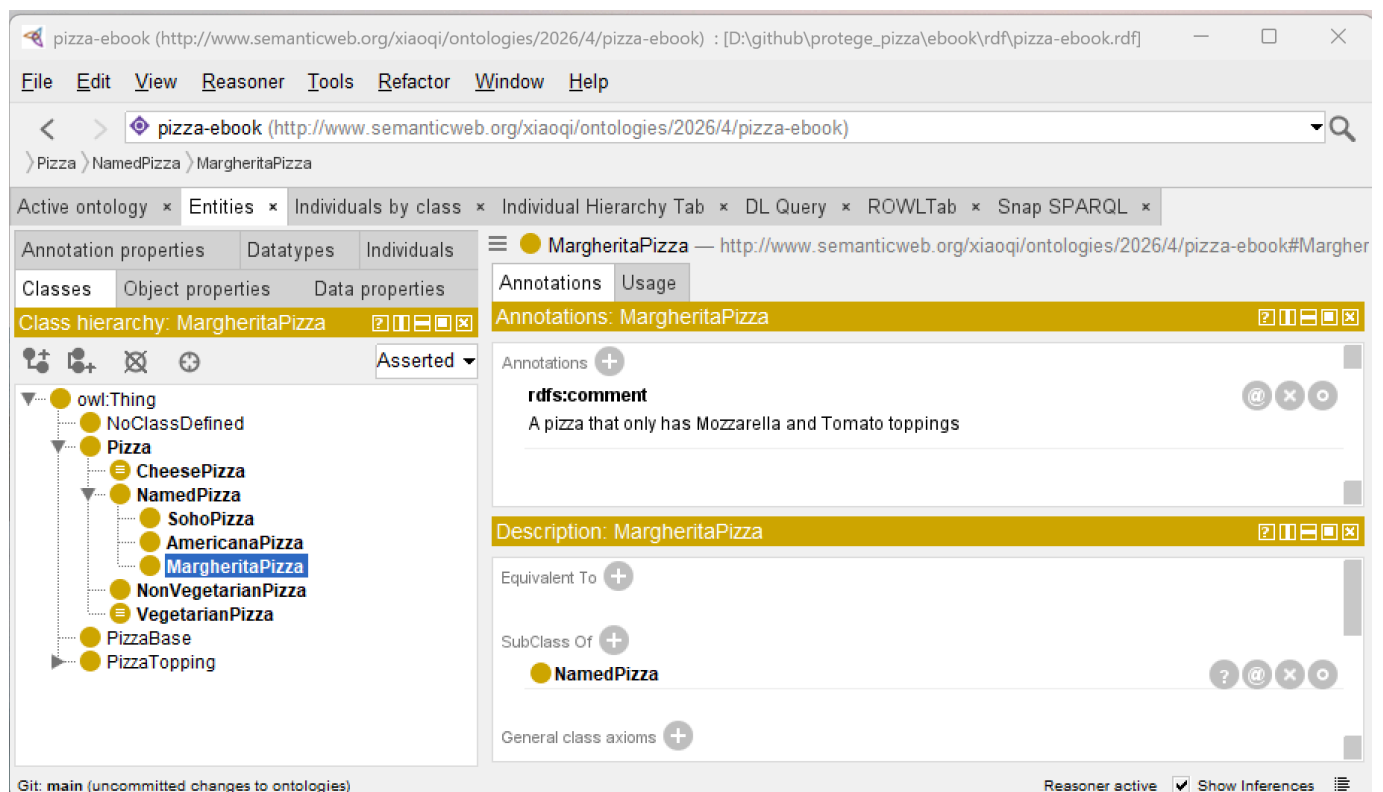
This abstraction improves ontology design in several ways.

- First, it avoids cluttering the top-level `Pizza` class with too many direct subclasses.
- Second, it creates a reusable semantic grouping for future reasoning and classification.
- Third, it improves maintainability as the ontology grows.

The tutorial then introduces `MargheritaPizza` as the first concrete named pizza.

This is especially important because `MargheritaPizza` becomes the first subclass that will later receive richer semantic definitions through restrictions.

After adding the `rdfs:comment` annotation, your ontology in Protégé should be looked as below:



At this stage, however, the subclass hierarchy only captures:

semantic specialization

rather than **full logical definition**.

In other words, Exercise 14 establishes the structural foundation upon which later chapters will build increasingly expressive semantic logic.

This reflects an important ontology engineering principle:

strong semantic reasoning requires strong semantic structure.

Before reasoning can become powerful, the ontology must first organize knowledge into meaningful conceptual hierarchies.

Exercise 14 therefore represents the beginning of ontology taxonomy engineering.

Finishing this exercise looks quite easy, you may pop up a quick question: how can I verify which subclasses I've really created in my ontology?

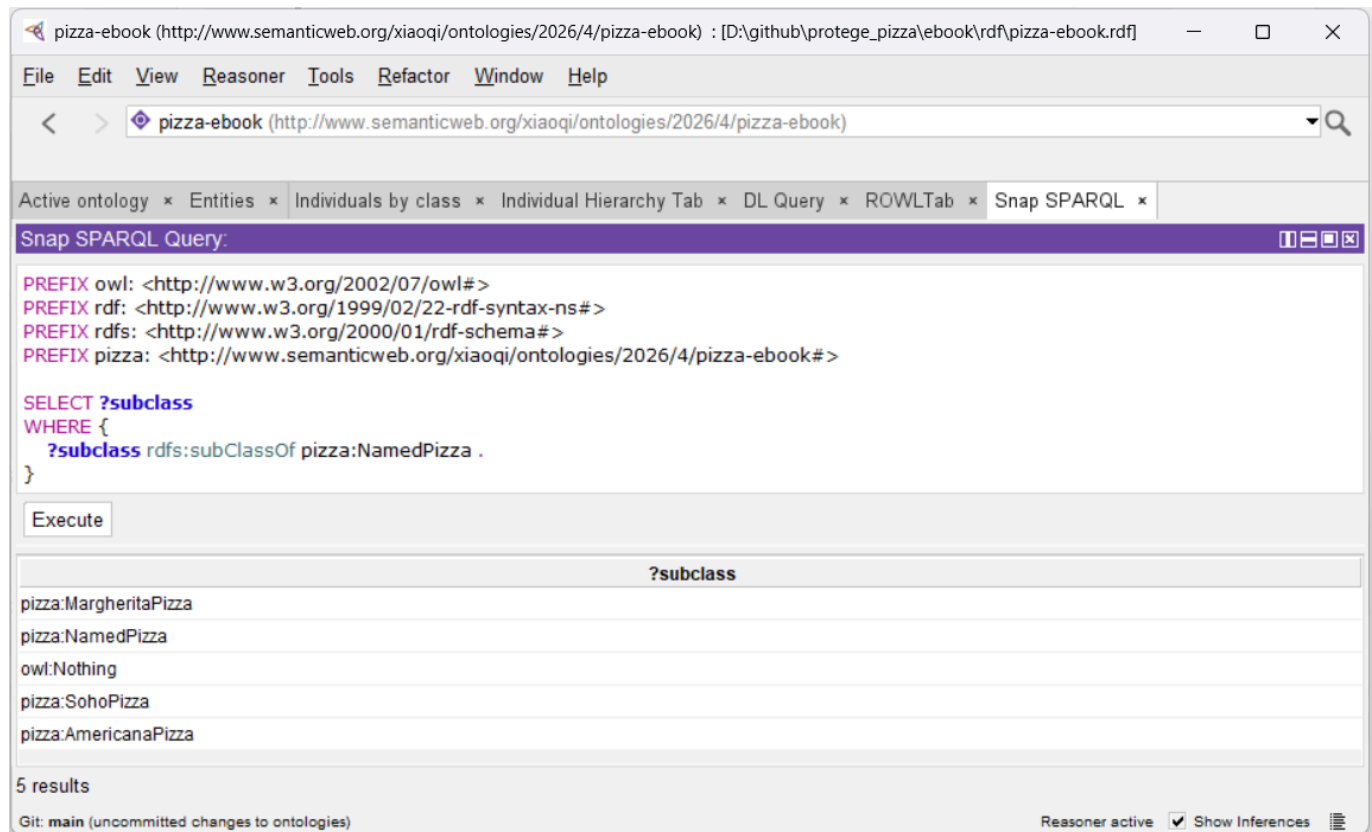
Valid question, instead of looking at Class tree structure hierarchy, the SPARQL is the tool for you to query such kind of needs. (For sure, SPARQL can do more!)

Below is the sample query to list all subclasses of **NamedPizza**:

```
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-scheman#>
PREFIX pizza: <http://www.semanticweb.org/xiaoqi/ontologies/2026/4/pizza-ebook#>

SELECT ?subclass
WHERE {
    ?subclass rdfs:subClassOf pizza:NamedPizza .
}
```

Result as below:



You can see all of those 3 specific named pizzas in the result, while `NamedPizza` itself is `subClassOf` `NamedPizza` as well, and also there's `owl:Nothing` as the subclass of any class, so total 5 results.

It's just for you to have a look and feel for now, no worry at this stage, we'll introduce and practice SPARQL in later chapters with more detail examples.

15.7 Semantic Inheritance

One of the most powerful consequences of subclass modeling is:

inheritance.

Inheritance means subclasses automatically receive semantic meaning from parent classes.

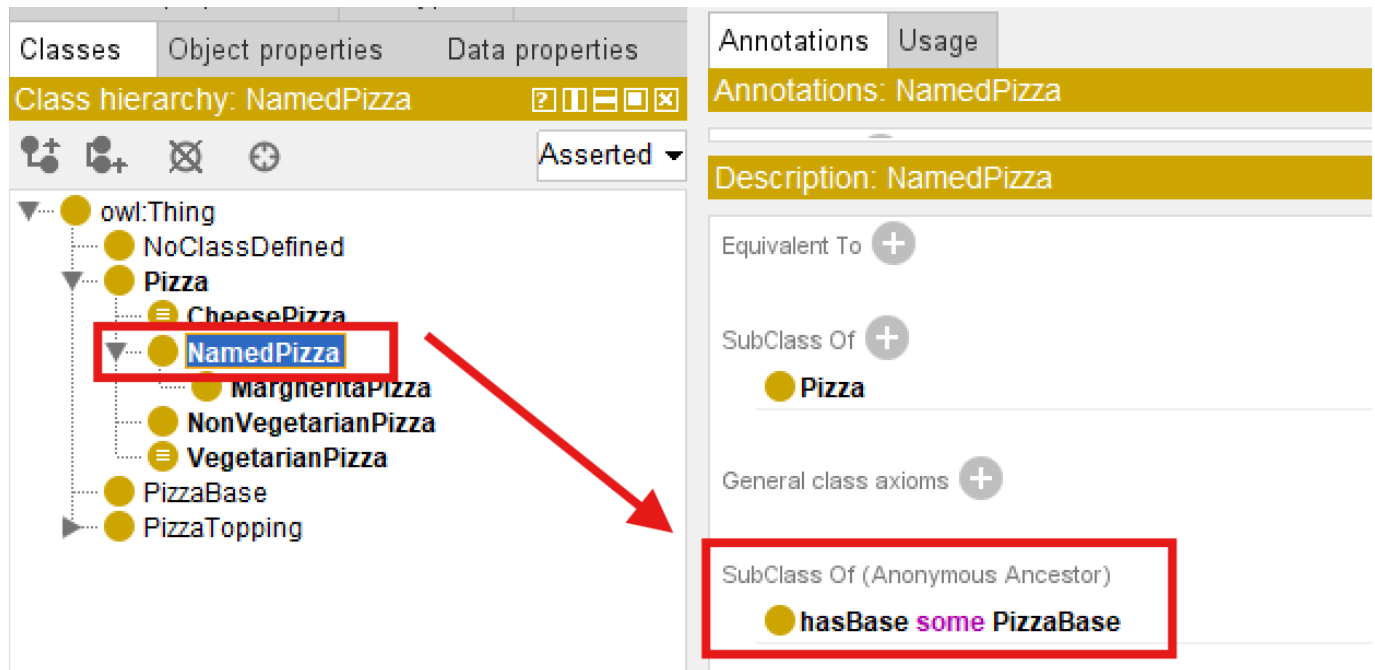
Support ontology defines: `Pizza SubClassOf hasBase some PizzaBase`

The screenshot shows a Semantic Web editor interface. On the left, the 'Classes' tab is active, displaying a 'Class hierarchy: Pizza'. The hierarchy starts with 'owl:Thing' at the root, followed by 'NoClassDefined', and then 'Pizza' (highlighted with a red box). Under 'Pizza', there are several subclasses: 'CheesePizza', 'NamedPizza', 'MargheritaPizza', 'NonVegetarianPizza', and 'VegetarianPizza'. Below these are 'PizzaBase' and 'PizzaTopping'. A red arrow points from the 'Pizza' class in the hierarchy to the 'Description: Pizza' panel on the right. In this panel, the 'SubClass Of' section is highlighted with a red box, showing that 'Pizza' is a subclass of 'hasBase some PizzaBase'.

Now consider: `NamedPizza SubClassOf Pizza`

This screenshot shows the same Semantic Web editor after adding the 'NamedPizza SubClassOf Pizza' axiom. The 'Class hierarchy: Pizza' panel on the left now includes 'NamedPizza' as a direct subclass of 'Pizza'. The 'NamedPizza' class is highlighted with a blue box. The rest of the hierarchy remains the same, with 'Pizza' still being a subclass of 'NoClassDefined', which is a subclass of 'owl:Thing'.

Then `NamedPizza` automatically inherits: `hasBase some PizzaBase`



No additional modeling is required.

This greatly reduces redundancy.

Inheritance enables ontologies to scale efficiently while preserving consistency.

This principle becomes increasingly important in enterprise knowledge graphs.

15.8 Interesting Reading -- Semantic Inheritance vs OOP Inheritance

Learners with software engineering backgrounds may notice that subclass inheritance in ontology appears similar to inheritance in object-oriented programming (OOP).

For example, in Java or C#, one might write:

```
class Pizza {...}
class NamedPizza extends Pizza {...}
```

This resembles OWL subclass definitions:

```
NamedPizza SubClassOf Pizza
```

At a superficial level, both seem to describe:

inheritance.

However, semantically, these two forms of inheritance are fundamentally different.

15.8.1 Inheritance in Object-Oriented Programming

In OOP, inheritance primarily supports:

code reuse and behavioral extension.

A child class inherits:

- attributes
- methods, and
- implementation behavior

For example:

```
./
├── Pizza
│   ├── size
│   ├── bake()
│   └── slice()
```

A subclass automatically gains executable behavior.

Inheritance in OOP is therefore closely tied to:

implementation.

The main question becomes:

What functionality can this class reuse or override?

15.8.2 Semantic Inheritance in Ontology

OWL inheritance operates differently.

Ontology classes do not contain executable methods.

Instead, they contain:

semantic meaning.

So, when a subclass inherits from a parent class, it inherits:

- class membership semantics
- logical constraints
- property restrictions
- reasoning implications

For example:

```
Pizza SubClassOf hasBase some PizzaBase
NamedPizza SubClassOf Pizza
```

This means **NamedPizza** inherits a semantic requirement:

hasBase some PizzaBase

No code executes.

No method is invoked.

Instead, the ontology inherits:

logical meaning.

15.8.3 Key Difference

The core distinction can be summarized as follows:

Aspect	OOP Inheritance	OWL Semantic Inheritance
Purpose	Code reuse	Meaning reuse
Inherits	Methods / fields	Constraints / semantics
Execution	Runtime behavior	Reasoning
Main Goal	Software extensibility	Knowledge inference

This distinction is especially important when moving toward knowledge graph systems.

Many engineers initially assume ontology classes behave **like** programming classes.

They do not!

A programming class describes:

how software behaves.

An ontology class describes:

what something means.

This difference becomes central in EKA.

K - Knowledge graphs do not become intelligent merely because they store objects.

They become intelligent when semantic meaning supports automated reasoning.

This is why semantic inheritance is fundamentally closer to:

mathematical set inclusion

than to software inheritance.

Ontology engineers therefore must gradually shift their mindset --

from thinking in terms of:

objects and methods

toward thinking in terms of:

concepts and logic.

15.9 Subclass vs. Instance -- Common Beginner Confusion

One of the most common beginner mistakes is confusing **subclasses** with **instances**.

Consider: **MargheritaPizza**

Should it be modeled as a class? Or as an individual?

The answer depends on intent.

If it represents:

a recipe category,

then it should be modeled as:

a subclass.

If it represents:

one specific pizza one a table,

then it should be modeled as:

an individual.

This distinction is critical.

Ontology engineers must constantly ask:

Am I modeling a category or a concrete entity?

Correct semantic modeling depends on this decision.

Besides Michael's **Pizza.owl** tutorial, another official document worth to read in detail is the **Ontology 101** (https://protege.stanford.edu/publications/ontology_development/ontology101.pdf), authored by Natalya F. Noy and Deborah L. McGuinness.

I learned quite insightful distinguish between an instance or a class from its Chapter 4.6, and quote following criteria for your reference:

Individual instances are the most specific concepts represented in a knowledge base.

If concepts form a natural hierarchy, then we should represent them as classes (subclasses).

15.10 Graph Database Perspective -- Hierarchy in Neo4j vs. Ontology

Subclass modeling also highlights an important distinction between **graph database** and **ontologies**.

In Neo4j, hierarchy may be stored as graph relationships:

```
CREATE (n:Class {name:"NamedPizza"})
CREATE (p:Class {name:"Pizza"})
CREATE (n)-[:SUBCLASS_OF]->(p)
```

This stores hierarchy structurally.

However, Neo4j itself does NOT automatically understand:

set inclusion semantics

unless EXPLICIT rules are implemented.

OWL reasoners behave differently.

They interpret subclass relationships logically.

This means:

- Graph database stores hierarchy.
- Ontology reasons over hierarchy.

This distinction is essential for enterprise architects building semantic systems.

15.11 EKA Perspective -- Taxonomy as Semantic Compression

From the perspective of Executable Knowledge Architecture (EKA):

$$EKA = (K, R, \Theta, \Phi, \Gamma),$$

subclass hierarchies strengthen multiple layers.

***K* - Knowledge Graph Layer**

Subclass hierarchy enriches semantic graph structure and conceptual organization.

***R* - Reasoning & Rules Layer**

Inheritance enables automatic classification and rule propagation.

Γ - Governance Layer

Taxonomy standardizes classification boundaries and semantic consistency.

A useful way to understand subclass hierarchy is as:

semantic compression.

Instead of repeatedly defining semantics for every concept, ontology allows shared semantics to be inherited through hierarchy.

This dramatically improves scalability.

Taxonomy therefore acts as a compression mechanism for semantic knowledge.

15.12 Key Concepts

Concepts	Description
Class	Semantic category in ontology
Subclass	Specialized class derived from parent (still representing the category)
Taxonomy	Hierarchical semantic structure
is-a Relationship	Automatic specialization relationship
Semantic Inheritance	Automatic inheritance of parent semantics
Set Inclusion	Mathematical interpretation of subclass
Classification	Assignment of entities to categories
Semantic Compression	Reuse of semantics through hierarchy

15.13 Chapter Summary

In this chapter (15), we introduced subclass creation as one of the foundational operations in ontology engineering.

Although creating subclasses in Protégé appears simple, subclass modeling carries deep semantic meaning.

You learned that subclasses enable:

- inheritance,
- specialization,
- reasoning, and
- semantic scalability.

Most importantly, subclass hierarchy transforms isolated concepts into structured semantic knowledge.

15.14 Protégé Skills Checklist

By completing this chapter (15), you should be able to:

- ☒ Create subclasses in Protégé
- ☒ Understand subclass semantics
- ☒ Distinguish subclass from instance
- ☒ Explain inheritance
- ☒ Understand taxonomy structure
- ☒ Explain subclass reasoning

15.15 Looking Ahead

In the next chapter (16), we will continue expanding the **Pizza** hierarchy by creating additional named subclasses and enriching their semantic meaning.

As the hierarchy grows, the ontology will become increasingly capable of supporting:

- richer classification,
 - stronger reasoning, and
 - more expressive semantic intelligence
-

Last updated at: 2026-06-29